

CS439: Luxury Beauty Recommendation System

Chiraayu Dedhiya

Rutgers University

ckd41@scarletmail.rutgers.edu

Divy Patel

Rutgers University

dp1206@scarletmail.rutgers.edu

ABSTRACT

Recommendation systems are integral to modern online platforms, enhancing user experiences through personalized suggestions. This project utilizes the Amazon 5-core dataset for the "Luxury Beauty" category to develop a recommendation system based on Matrix Factorization. The model captures latent relationships between users and items, integrating personalized user embeddings and item-specific features to predict preferences accurately. Other evaluation metrics analyze the system's effectiveness, including RMSE, MAE, Precision, Recall, F-measure, and NDCG. Parameter tuning and regularization ensure optimized performance. The recommendations produced by the system highlight their relevance and diversity, emphasizing the effectiveness of latent factor models in personalized recommendation tasks. The report includes a comprehensive analysis of the methodology, findings, and implications.

KEYWORDS — luxury beauty recommendation, matrix factorization, collaborative filtering, and explainability

I. INTRODUCTION

Recommendation systems play a crucial role in online platforms, helping users discover relevant content and improving their overall experience. The success of these systems often depends on the quality of the dataset used. To address issues like sparsity, datasets are often processed into dense subsets, such as k-core datasets. A k-core dataset ensures that every user and item has a minimum number of interactions, which makes the data more robust for training models. For this project, we used the Amazon 5-core dataset for the "Luxury Beauty" category, focusing on a ratings-only version that includes user-item-rating-timestamp tuples. This simplified dataset allows us to efficiently build and test recommendation algorithms.

We implemented a Matrix Factorization model, which is a model-based collaborative filtering method. Unlike memory-based collaborative filtering, which relies on user or item similarity, Matrix Factorization captures latent relationships by decomposing the user-item interaction matrix into lower-dimensional vectors (embeddings). This approach learns user and item features that can predict ratings effectively while incorporating user, item, and global biases to enhance accuracy. Using PyTorch as a toolkit, we followed the recommended practice of designing the model from scratch rather than relying on pre-built algorithms. By preprocessing the data and carefully tuning the model, we were able to generate accurate and

personalized recommendations. The project demonstrates the effectiveness of Matrix Factorization for real-world recommendation tasks.

II. RELATED WORK

A. Matrix Factorization

Matrix factorization techniques have revolutionized recommender systems by effectively capturing latent user-item interactions through decomposition of the interaction matrix. These methods gained prominence during the Netflix Prize competition, where they outperformed traditional approaches by incorporating implicit feedback and temporal dynamics. Koren et al. (2009) highlighted the scalability and flexibility of matrix factorization, particularly in handling implicit feedback and systematic biases. Enhancements such as regularization and hybrid models combining collaborative and content-based methods have further improved accuracy. These advancements have established matrix factorization as a cornerstone of modern recommendation systems, widely adopted across domains like e-commerce and streaming platforms.

B. Collaborative Filter

Collaborative filtering (CF) is a widely used approach in recommender systems. Huang et al. (2015), categorize it into memory-based and model-based methods. They highlighted the role of advanced similarity measures and hybrid approaches in improving the accuracy and scalability of these systems, particularly under constraints like data sparsity and cold-start challenges. Despite these improvements, challenges like data sparsity, scalability, and cold-start problems persist, driving ongoing research to refine CF methods and enhance their effectiveness across diverse applications.

C. Algorithm Design

Thirumal, Namitha, and Agnus explore various algorithms used in recommendation systems, focusing on collaborative filtering methods such as Singular Value Decomposition (SVD) and Pearson's correlation. It highlights the significance of collaborative filtering in refining data and predicting user preferences by identifying relationships between users and items. Through a comparative analysis, the study demonstrates that SVD outperforms Pearson's correlation in terms of accuracy.

III. PRELIMINARIES

The dataset used in this study is sourced from the publicly available Amazon 5-core dataset, a "Small subsets for experiment" collection designed to facilitate efficient analysis while maintaining data richness. The 5-core dataset ensures that every user and item in the dataset has a minimum of five

associated reviews, reducing data sparsity and making it well-suited for building recommendation models. The dataset can be downloaded from the official website, where detailed documentation and examples are provided.

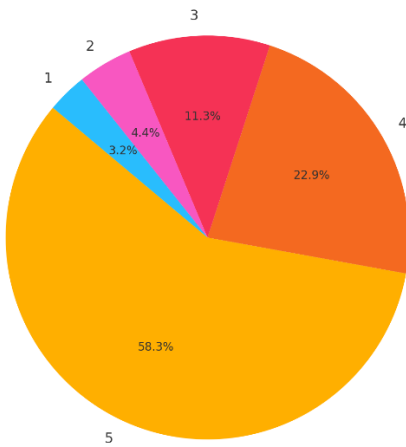
Each entry in the dataset represents a user-item interaction and include several key fields

- reviewerID: The unique identifier for the user.
- asin: The unique identifier for the product.
- overall: A 1-5 integer star rating provided by the user for the product.
- reviewText: The textual content of the user's review.
- summary: The title of the review.
- timestamp: The time the review was submitted.
- helpfulness: Two numbers indicating the number of users who found the review helpful or unhelpful.
- Image: A 4096-dimensional vector representing the product's image, derived using a convolutional neural network.
- Metadata: Additional product information such as title, price, brand, category, and image URL.

For this study, we focus only on a subset of the fields: reviewerID, asin, overall, and reviewText. These fields provide the user ID, product ID, rating, and review content, respectively, which are sufficient for building and evaluating our recommendation model. The dataset was initially provided as a JSON file containing a total of 34,278 reviews. To prepare the data for modeling, it was split into training and testing sets, with 80% of the reviews (27,422 entries) used for training and the remaining 20% (6,856 entries) reserved for testing. This stratified split ensured that user-specific information was appropriately distributed between the training and testing sets, maintaining consistency for fair evaluation. The processed datasets were saved as `luxury_beauty_train.csv` and `luxury_beauty_test.csv` for further analysis.

To get an idea of what consists within the dataset:

Distribution of Ratings Amongst Luxury Brands



IV. PROBLEM FORMALIZATION

This project is to develop a recommendation system that predicts a user's rating for unseen items based on their interaction history. Mathematically, the problem can be formalized as follows:

A. Objective

Given a set of users $U=\{u_1, u_2, \dots, u_m\}$, a set of items $I=\{i_1, i_2, \dots, i_n\}$, and a matrix R of observed user-item ratings, where $R_{u,i}$ represents the rating given by user u to item i . The goal is to predict the entries $\widehat{R}_{u,i}$ in R . These predictions will be used to generate a ranked list of top-10 recommendations for each user.

B. Input

C. A set of observed ratings $R_{train} = \{R_{u,i} \mid u \in U, i \in I\}$ where each rating $R_{u,i} \in [1,5]$

Optional textual reviews $T=\{T_{u,i}\}$, where $T_{u,i}$ is the review content provided by user u for item i .

D. Model Formulation

Matrix Factorization is a model-based collaborative filtering technique used to predict missing values in a user-item interaction matrix R . The formula decomposes the interaction matrix R into two lower-dimensional matrices P and Q .

$$R_{m \times n} = P_{m \times k} * Q_{k \times n}^T$$

R = user item interaction matrix with dimensions $m \times n$

P = user latent feature matrix of dimensions $m \times k$

Q = item latent feature matrix of dimensions $n \times k$

k = number of latent factors capturing the relationships between users and items

Predicted rating $\widehat{R}_{u,i} = p_u^T q_i + b_u + b_i + \mu$

p_u = latent vector for user u

q_i = latent vector for item i

b_u = user bias

b_i = item bias

μ = average rating across all users and items

Matrix factorization model: $\widehat{R} = P * Q^T + b_u + b_i + \mu$

E. Root Mean Square Error

Root mean square error measures the standard deviation of residuals, providing an indication of the absolute fit of a model to the data. It reflects how closely the observed data points align with the model's predicted values. RMSE is a widely used metric to quantify the differences between predicted and observed values. A lower RMSE value indicates better model performance, signifying that the predicted values are closer to the observed ones, thereby achieving a closer approximation to the line of best fit. RMSE is calculated as the square root of the mean of the squared errors, where larger errors contribute more significantly due to the squaring operation, amplifying their effect on the final RMSE value.

$$\text{RMSE} = \sqrt{\left(\frac{1}{n}\right) \sum_{i=1}^n (y_i - x_i)^2}$$

n = number of data points

y_i = predicted value

x_i = observed value

F. E. Mean Absolute Error

The mean absolute error measures the average magnitude of the errors in the set of predictions. The MAE is a linear score which means that all the individual differences are weighted equally in the average. It tells us how big of an error we can expect from the prediction model on average. The MAE will always be lesser than or equal to RMSE. The lower the value of MAE, the higher is the accuracy of the prediction model.

$$\text{MAE} = \left(\frac{1}{n}\right) \sum_{i=1}^n |y_i - x_i|$$

n = number of data points

y_i = predicted value

x_i = observed value

F. Precision

Precision at k measures the proportion of relevant items in the top- k recommendations. It is the fraction of relevant items within the top- k set. For instance, if the precision for a top 10 recommendation model is 70%, it means that 70% of the items recommended by the model are relevant to the user. Higher precision values indicate a more effective recommendation model

$$\text{Precision} = \frac{|\text{Recommend Items that are relevant}|}{|\text{Recommended Items}|}$$

Recommended items are those items which have predicted rating greater than or equal to threshold.

G. Recall

Recall at k is the proportion of relevant items found in the set of top k recommendations. For example, if the recall at 10 is calculated to be 45% for our prediction model, then 45% of the total number of relevant items appear in the top 10 recommendations.

$$\text{Recall} = \frac{|\text{Number of relevant items retrieved}|}{|\text{Total number of relevant items}|}$$

Relevant items are those items which have original rating greater than or equal to threshold

H. F-Measure

F-measure is a measure of the production model's accuracy. F-measure ranges from 0 to 1, where a value closer to 1 indicates better model performance.

$$F - \text{measure} = \frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

I. Normalized Discounted Cumulative Gain (NDCG)

NDCG is a ranking metric that uses relevance scores in the form of real numbers to evaluate the quality of a recommendation list. It measures how well the list prioritizes relevant items, with the idea that more relevant items should appear at the top since users are more likely to notice and interact with these. Each item in the recommendation list is assigned a relevance score, which in our case is the original rating. This relevance score is referred to as the gain. To see the most relevant items at the top of the list, we divide each by the logarithm of the item position, which is called discounted cumulative gain.

$$\text{DCG} = \sum_{i=1}^n \frac{rel_i}{\log_2(i+1)}$$

To ensure DCGs are directly comparable across users, they need to be normalized. This is done by arranging all items in the optimal order based on their actual ratings and calculating the DCG for this ideal ranking, referred to as the Ideal DCG (IDCG). The raw DCG is then divided by the IDCG to compute NDCG.

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}$$

V. THE PROPOSED MODEL

The required tasks involve selecting a suitable dataset, splitting it into 80% training and 20% testing sets, and preprocessing the data, including reviews. Using the training data, a model (e.g., Matrix Factorization) must predict ratings for the testing set and generate a top-10 recommendation list for each user, avoiding items already rated in the training set, with evaluations based on metrics like MAE, RMSE, and precision. The recommendation system implemented in this project relies on a Matrix Factorization model, a popular collaborative filtering approach. This model decomposes the user-item interaction matrix into low-dimensional representations (embeddings) for users and items.

A. A. Data Selection and Preprocessing

1. Process Overview: We selected the Amazon 5-core dataset for "Luxury Beauty" due to its manageable size and relevance to the task.

2. Key Components

- **Importing Libraries:** Using pandas and train_test_split from sklearn for data processing. The pandas library is used for data manipulation and analysis. The train_test_split function from sklearn splits the dataset into training and testing.
- **Extracting Relevant Fields:** Extracts the necessary fields: reviewerID, asin, overall, and reviewText. These fields correspond to the user ID, product ID, user rating, and review text, respectively.
- **Splitting the Dataset:** The dataset was then split into 80% training and 20% testing sets to develop our model.

B. B. Rating Prediction

1. Process Overview: The rating prediction process begins by loading preprocessed training and testing datasets and mapping user and item IDs to numerical indices. A matrix

factorization model is defined using PyTorch, including user/item embeddings, biases, and a global bias. The model predicts ratings based on latent interactions and optimizes parameters using Mean Squared Error loss with L2 regularization, via the Adam optimizer. The trained model is saved as `matrix_factorization_model.pth`

2. Key Components

- **Importing Libraries:** Rating prediction process begins with the import of essential libraries, including torch for implementing the matrix factorization model, pandas for data handling, and sklearn.metrics for evaluating model performance.
- **Loading and Mapping Data:** The training dataset is loaded into pandas dataframes. To facilitate model training, unique user and item IDs are mapped to numerical indices. This mapping ensures compatibility with the embedding layers of the matrix factorization model.
- **Matrix Factorization Model:** The matrix factorization model is defined using PyTorch, and incorporates user and item embeddings, and user and item biases. Each user and item is represented by a latent vector in k -dimensional space, where $k=300$ (embedding size). These embeddings capture latent relationships and interactions between users and items. Embeddings are initialized with normal distribution and biases are initialized to zero. Predicted rating computed as: $\widehat{R}_{u,i} = p_u^T q_i + b_u + b_i + \mu$. The model computes regularization loss to reduce overfitting.
- **Optimization, Loss Function, and Training Model:** The model minimizes Mean Squared Error (MSE) loss, a commonly used metric for regression tasks. The MSE loss is defined as: $L_{MSE} = \frac{1}{N} \sum_{(u,i) \in R_{train}} (R_{u,i} - \widehat{R}_{u,i})^2$ where $R_{u,i}$ is actual rating provided by user for item. $\widehat{R}_{u,i}$ is predicted rating. To prevent overfitting, L2 regularization is added $L = L_{MSE} + \lambda(|p_u|^2 + |q_i|^2 + b_u^2 + b_i^2)$ where $\lambda=0.03$ is the regularization parameter. Adam optimizer is used to minimize the loss function and update the model parameters. using a learning rate of 0.0005 and a batch size of 128. The training process involves mini-batches to calculate gradients for subsets of data, to improve speed. For each mini-batch, the model computes predictions, calculates the combined loss, and updates parameters. The model is trained over 50 epochs, with loss monitored after each epoch to ensure stability.
- **Evaluation:** The model's performance is evaluated on the testing dataset after training to measure its accuracy in predicting user ratings. Two key metrics: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).
- **Saving the Trained Model:** Once the training is complete, the model's learned parameters, including user and item embeddings, biases, and the global bias, are saved using PyTorch's `torch.save` function. For future use, the model is stored as `matrix_factorization_model.pth`.

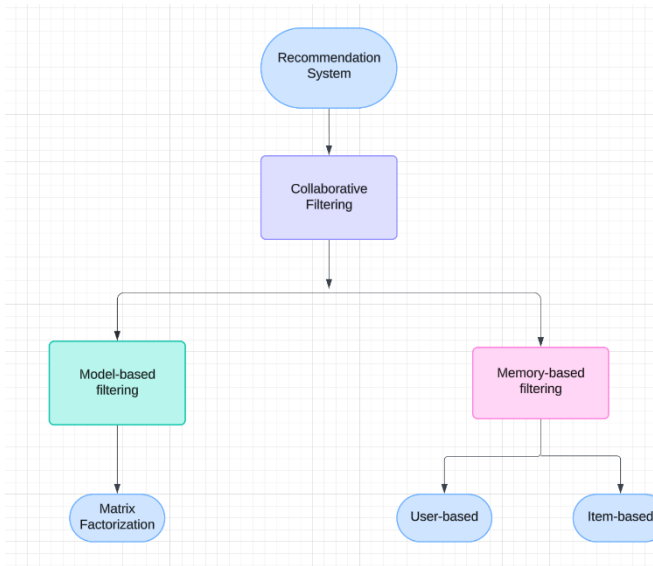
C. B. Item Recommendation

1. **Process Overview:** The item recommendation process utilizes the trained matrix factorization model to generate personalized recommendations for each user. For test data set the model predicts ratings, ranks these items based on their scores, and selects the top 10 for recommendation. The recommendation lists are evaluated using metrics such as Precision, Recall, F1-Score, and NDCG.

2. Key Components

- **Prediction and Rating:** The predicted rating for each user-item pair is calculated using the formula for $\widehat{R}_{u,i}$. Once predicted, the ratings are sorted in descending order, prioritizing items with the highest scores. By generating recommendations in the test set, ensures that each user receives a concise and personalized list of recommendations tailored to their predicted preferences.
- **Filtering:** The system filters out items that users have already rated in the training set. This is achieved by maintaining a record of items rated by each user during training and excluding these from the pool of candidate items. This filtering step guarantees that the recommendations consist only of new and unseen items. Threshold is applied to the predicted ratings to filter out low-scoring items. Only items with predicted scores above the threshold are considered for the final recommendation list.
- **Top 10 Recommendation List:** The top 10 list is personalized for each user and represents the items with the highest predicted ratings.
- **Evaluation Metrics:** To assess the quality of the recommendation lists, the system uses standard metrics like Precision, Recall, F1-Score, and NDCG. Precision measures the proportion of recommended items that are relevant, while Recall evaluates the fraction of all relevant items successfully included in the recommendations. The F1-Score combines these two metrics to balance accuracy and completeness. Additionally, NDCG (Normalized Discounted Cumulative Gain) evaluates the ranking quality of the recommended items. These metrics are calculated in the code to provide a comprehensive evaluation of the system's performance, ensuring that the recommendations are accurate.

Overview of Recommendation System



VI. EXPERIMENTS

The experimental results assess the matrix factorization model using the Amazon 5-core "Luxury Beauty" dataset. Rating prediction was evaluated using RMSE and MAE, showing accurate convergence after 50 epochs. For top-10 recommendations, metrics such as Precision, Recall, F1-Score, and NDCG demonstrated the model's ability to deliver suggestions. The results validate the effectiveness of the model.

Metric	Value
MAE	0.6891
RMSE	0.9199
Precision	0.8041
Recall	0.7361
F-measure	0.7595
NDCG	0.8041

VII. TRANSPARENCY AND EXPLAINABILITY OF RECOMMENDER SYSTEMS

1. Process Overview: The Explainability module was trained to enhance transparency and trust by breaking down the predicted ratings into interpretable components. This approach will allow users to understand the logic behind recommendations, making the system more reliable and accountable for its recommendations to the users.

2. Key Components

- **Interaction Score:** Measures the alignment between user preferences and item attributes. It quantifies how well a specific item matches the user's tastes based on learned embeddings from model training.
- **User Bias:** Represents the individual tendencies of users, such as their general tendency to rate items higher or lower than average.

- **Item Bias:** Reflects item-level factors like popularity or quality that may influence the ratings independently of the user based on their previous preferences.
- **Global Bias:** Captures the dataset-wide average rating to provide a baseline for recommendations to the user.

3. Functionality

- When a recommendation is made, the predicted rating is broken into these four components. For example, in a recommendation for a luxury beauty product with a predicted rating of 3.6996:

Interaction Score	0.0000
User Bias	0.2872
Item Bias	0.4811
Global Bias	2.9313

VIII. FAIRNESS AND UNBIASES OF RECOMMENDER SYSTEMS

1. Process Overview: The Fairness module ensures equitable and equal recommendations by addressing biases ratings across demographic groups and product categories. This module was designed essential for maintaining user trust and preventing algorithmic discrimination by using specific category ratings.

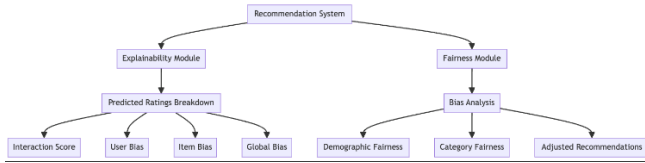
2. Key Components

- **Demographic Fairness:** Analyzes biases in recommendations across different user demographics (e.g., gender, age) and ensures balanced outcomes.
- **Category Fairness:** Evaluates bias across product categories, ensuring that certain categories are not disproportionately favored.
- **Bias Mitigation:** Dynamically adjusts recommendation weights to balance biases identified during evaluation of the ratings to ensure fairness

3. Functionality : When a recommendation is made, the Fairness module evaluates and mitigates biases in the system to ensure equal outcomes. For example, in the case of a luxury beauty product recommendation:

- **Demographic Fairness:** 2.0623 was calculated for an unspecified demographic group.
- **Category Fairness:** 3.1876 was calculated for unclassified product categories.
- **Bias-Mitigated Adjustments:** The recommendation system dynamically adjusted the score for the product, resulting in an overall bias-mitigated score of 4.1370.

Overview of Other Recommendation System



IX. CONCLUSION AND FUTURE WORK

In this project, we developed a recommendation system using Matrix Factorization on the Amazon 5-core "Luxury Beauty" dataset. The model effectively captured latent user-item interactions, predicting ratings and generating personalized top-10 recommendations. Evaluation metrics revealed a satisfactory performance, with MAE at 0.6891, RMSE at 0.9199, Precision at 0.8041, Recall at 0.7361, F-measure at 0.7595, and NDCG at 0.8041. These results highlight the system's ability to produce relevant recommendations, but some limitations were observed. The relatively lower Recall indicates that not all relevant items were included in the top-10 recommendations, and the prediction errors reflected in MAE and RMSE suggest potential improvements. These shortcomings can be attributed to the dataset's sparsity, the exclusion of additional metadata, and inherent biases in user ratings and item popularity.

To address these challenges, future work could incorporate richer features such as product categories, prices, and user demographics, as well as insights derived from sentiment analysis of review text. Tackling sparsity through hybrid

models that combine collaborative filtering and content-based approaches, along with leveraging implicit feedback like click-through rates, could enhance the model's robustness. Further, refining the fairness module with advanced debiasing techniques and adversarial training would ensure more equitable recommendations across demographic groups and product categories. Algorithm optimization, such as experimenting with Bayesian personalized ranking and fine-tuning parameters, may improve accuracy and ranking quality. Additionally, expanding the explainability module to provide deeper insights into user-item interactions, with user-facing visualizations, would enhance system. Finally, testing the system on larger, more diverse datasets and in real-world deployment scenarios would assess its scalability and generalizability.

ACKNOWLEDGEMENT

We sincerely thank our professor for their guidance, the teaching assistant for their support, and the graders for their time and feedback in evaluating our work.

REFERENCES

- [1] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 8 (2009), 30–37.
- [2] Ruisheng Zhang, Qi-dong Liu, Chun-Gui, Jia-Xuan Wei, and Huiyi-Ma. 2014. Collaborative Filtering for Recommender Systems.
- [3] Aamir Fareed, Saima Hassan, Samir Brahim Belhaouari, Zahid Halim, A collaborative filtering recommendation framework utilizing social networks, *Machine Learning with Applications*, Volume 14, in *Computer*, vol. 12, no. 2, pp. 66-78, Feb. 1979, doi: 10.1109/MC.1979